

Decision tree algorithm - Gini impurity and implementation with scikit-learn

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain how a decision tree makes rule-based predictions without learning weight parameters.
- Compute Gini impurity by hand for a node and for a candidate split.
- Build a scikit-learn implementation of a `DecisionTreeClassifier` and evaluate its plotted tree.
- Identify overfitting and underfitting in a tree and reason about the depth trade-off.

2. Detailed Explanation

Decision trees are rule-based, not learning-based

A decision tree is a classification algorithm that builds a nested sequence of if-else conditions directly from a training data set — it does not learn any weight parameters. Algorithms like logistic regression, linear regression, support vector machines (SVM), and neural networks are learning-based: they use an optimization algorithm to search for weight parameters (for example, $W_0, W_1, \dots, W_N$). A decision tree skips that step entirely. Instead, rules are derived directly by examining the data, and a sequence of those rules is followed to reach a prediction. Think of it as the same reasoning people use constantly in daily life: check one condition, then based on the outcome, check a further condition, and so on until a final decision is reached.

The two families differ in several practical ways:

Aspect	Decision tree	Logistic regression / SVM / neural networks
Learning mechanism	No optimization; rules derived from data	Optimization algorithm learns weight parameters
Description	Simple to trace exactly	Complex — relies on the shape of a fitted function

Computational cost	Less expensive	More expensive
Class capability	Multi-class by nature (like random forest)	Binary by default (logistic regression, SVM); neural networks are multi-class by nature
Explainability	Highly explanatory — the exact path can be shown to anyone	Hard to explain, especially to a non-technical person
Small-data performance	Can outperform learning-based algorithms on small data (tens to a few hundred instances)	"Hungry for data" — tends to win on huge data sets (thousands+)

Neither family is strictly superior. With a small data set, deriving good rules is easier than learning good weights, so a tree can win. With a huge data set, manually or algorithmically deriving rules for every case gets difficult, and learning-based algorithms tend to perform better. Random forest is a larger, tree-based ensemble built on the same principles as a single decision tree, and it is also multi-class by nature.

The everyday-life analogy: weather and going out to play

Decision trees mirror a decision-making pattern already used in daily life. Imagine deciding whether to go outside to play, based on the weather:

Weather is the first thing checked — the root. Without knowing the weather, no decision can be made.

Sunny → confident decision: go play.

Cloudy → in most cases, still go and play.

Raining → uncertain ("dicey"), so a further attribute is checked: the intensity of the rain.

Heavy rain → in most cases, do not go play.

Light rain / drizzling → in most cases, still go play.

This is a nested if-else: check the weather, and if it is raining, check a second condition (intensity). The final class value — play or not — is reached only after traversing the relevant conditions. In this example, weather is the input feature and play (yes/no) is the class value, making it a binary classification problem. Adding a second feature — intensity (encoded numerically, e.g. higher for sunny, lower/zero for light rain, higher again for heavy rain) — helps resolve the otherwise-confusing "raining" branch.

Anatomy of a decision tree

A tree is built from a small set of named parts:

Root node: the first node, corresponding to the most informative feature — the one that, when split on, gives the maximum reduction in impurity.

Branch / edge: the outcome of a yes/no (pass/fail) test on an attribute; a node proceeds either in the "yes" direction or the "no" direction.

Leaf node: the terminal node of a path, representing the final class value.

Depth / level: the root is level 0. Each additional branch traversed adds 1 to the depth. A tree with two levels below the root has depth 2.

Pure node: a node containing instances of only one class — no confusion about the predicted class there.

Tree construction repeatedly asks: "Which available attribute (with which threshold, for numeric attributes) gives the maximum reduction in impurity right now?" The first split uses the most informative feature; each later split uses the next most informative feature among those remaining, applied to the shrinking subset of data still unresolved at that branch. Ideally, at each split, roughly half the data set gets fully resolved into a pure node while the rest passes down for further splitting.

Watch out for: growing a tree is deliberately controlled. Construction does not continue indefinitely until every instance is perfectly classified — a maximum depth (or similar stopping criterion) is fixed in advance to keep the tree from growing without bound.

Multi-class nature of decision trees

Logistic regression and SVM are binary classifiers by nature and need deliberate manipulation to handle more than two classes. Decision trees (and random forest) are multi-class by nature instead — a tree node simply gets more branches for more classes. Extending the weather/play example: instead of only "go to play" (yes/no), add a third class such as "go do art" when the weather is cloudy and the person is not interested in playing. That node then has three branches instead of two. Neural networks are also multi-class by nature, unlike the other traditional learning-based algorithms.

Worked example: the umbrella decision tree

A second nested if-else example shows how a decision tree can capture everyday reasoning about carrying an umbrella:

Root node: Is it raining now?

Yes → carry an umbrella (leaf, class = "carry umbrella").

No → check a second feature: does the forecast say rain?

Yes (forecast says rain) → carry an umbrella.

No → skip the umbrella.

This tree has two classes ("carry umbrella," "skip it") and uses two features (current rain status, forecast). The first if-condition checks "is it raining now"; the second, nested inside the "no" branch, checks the "forecast" feature.

Worked example: study hours and exam pass/fail

This example reuses a data set previously analyzed with logistic regression, where the question was whether a student who studies for a given number of hours will pass or fail an exam. That earlier analysis found the pass probability crossed 0.5 at 4 study hours, so 4 hours is reused here as a natural first threshold. A second feature is added: whether the student attended a recovery class (a class offered before the exam to help students who missed the regular classes), encoded as 0 (did not attend) or 1 (attended).

The reconstructed 8-instance data set:

Hours of study	Attended recovery class	Result
2	No (0)	Fail
3	No (0)	Fail
2	Yes (1)	Fail
3	Yes (1)	Pass
5	—	Pass
6	—	Pass
5	—	Pass
6	—	Pass

The recovery-class value is left unmarked for the four students who studied 5 or 6 hours, because it doesn't change their outcome. Attendance was mixed among them, yet all four still passed — their study hours already comfortably exceeded the 4-hour threshold. Overall: 3 fail, 5 pass out of 8. Among the four students who studied fewer than 4 hours, two did not attend the recovery class and both failed. Of the two who did attend, one passed and one still failed — this subgroup remains genuinely mixed even after accounting for recovery-class attendance.

Building the tree by hand. The root split is "hours of study ≥ 4 ":

Right branch (hours ≥ 4): 4 instances, all pass $\rightarrow$ pure node, class = Pass.

Left branch (hours < 4): 4 instances, 3 fail / 1 pass → impure, needs a further split.

A student who studied 5 hours is compared against the root condition ($5 \geq 4$), so the tree assigns class Pass — this is how a trained tree classifies a brand-new instance using purely historical, already-learned rules.

The second split, applied only on the left branch, is "attended recovery class":

Left (attended = 0): 2 instances, both fail → pure node, class = Fail.

Right (attended = 1): 1 pass / 1 fail → still impure (50/50). This node is intentionally left unresolved here, to illustrate the depth trade-off.

At this point the tree has depth 2 (root at level 0, first split at level 1, second split at level 2). The one instance in the "attended = 1" leaf stays misclassified regardless of which class it is arbitrarily assigned, since it's a genuine 50/50 tie. A further feature, such as a student's past academic record, could resolve this last node into two pure nodes. Doing so would push the tree to depth 3 purely to correctly classify one additional instance, so this is deliberately avoided in favor of a shallower, "optimal" tree. A small number of misclassified instances is acceptable if it keeps the tree from growing unnecessarily deep.

Optimal tree: a tree that classifies the majority of instances correctly without being unnecessarily deep. It is not required to be 100% accurate on the training data. The goal is to balance depth (and therefore complexity) against accuracy, on the assumption that a tree performing well on training data will perform similarly on new (test) data.

Gini impurity

Gini impurity (Gini index) is the default metric scikit-learn uses to decide how to split a tree. For a node with two or more classes:

$$\text{Gini} = 1 - \sum p(\text{class}_i \mid \text{branch})^2$$

Here the sum runs over all classes present in the branch, and $p(\text{class}_i \mid \text{branch})$ is the proportion of instances in that branch belonging to class i . A Gini of 0 means the node is completely pure (one class only). The maximum Gini for binary classification is 0.5, occurring at a perfectly mixed 50/50 node where nothing is known yet.

Applying the formula to the study-hours example:

Root node (3 fail / 5 pass out of 8): $\text{Gini_root} = 1 - (3/8)^2 - (5/8)^2 = 0.469$.

Split 1 — "hours ≥ 4 ":

Right branch (4 pass): $\text{Gini_right} = 1 - (4/4)^2 = 0.0$

Left branch (3 fail / 1 pass): $\text{Gini_left} = 1 - (3/4)^2 - (1/4)^2 = 0.375$

Weighted Gini after the split: $(4/8) \times 0.375 + (4/8) \times 0.0 = 0.1875$ (≈ 0.188)

Reduction in impurity from the root: $0.469 - 0.188 = 0.281$.

Split 2 (on the left branch of split 1) — "attended recovery class":

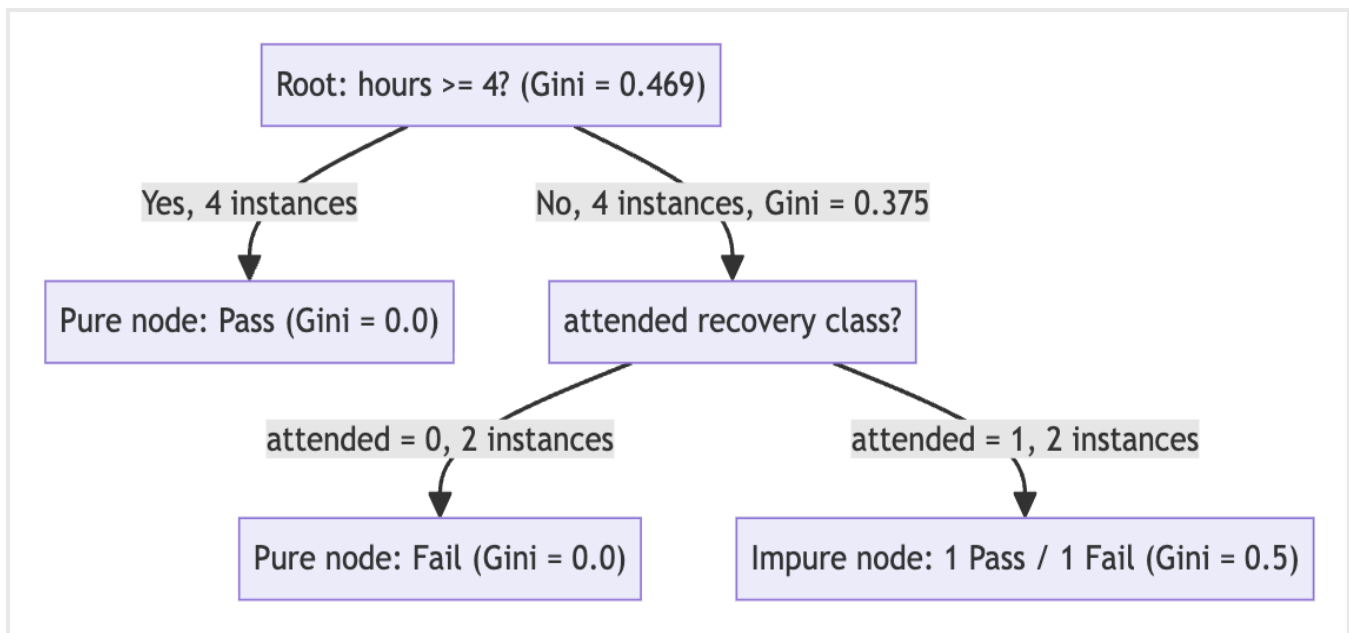
Left (attended = 0, 2 fail): $\text{Gini} = 1 - (2/2)^2 = 0.0$

Right (attended = 1, 1 pass / 1 fail): $\text{Gini} = 1 - (1/2)^2 - (1/2)^2 = 0.5$

Weighted Gini for this split: $(2/4) \times 0.0 + (2/4) \times 0.5 = 0.25$

If "attended = 1" alone had been tried as the very first split on the whole 8-instance data set, the resulting weighted Gini would be 0.438 — far worse than the 0.188 achieved by splitting on "hours $\geq$ 4" first. Because "hours $\geq$ 4" gives a much larger reduction in impurity, it becomes the root-node condition, and "attended recovery class" is used only as the second-level split.

The general rule for choosing any split: among all candidate attributes (and, for numeric attributes, candidate thresholds), pick the one whose split produces the lowest weighted Gini — equivalent to picking the greatest reduction in impurity relative to the parent node's Gini. Thresholds like "hours $\geq$ 3," "hours $\geq$ 5," or "hours $\geq$ 6" could also be tried. "Hours $\geq$ 4" and "hours $\geq$ 5" happen to give the same split outcome here, but "hours $\geq$ 6" would wrongly exclude one instance that should be on the pass side. In practice, with larger data sets, code evaluates the reduction in impurity across many candidate thresholds automatically, rather than guessing by eye.



Decision boundary visualization

Each split can be pictured as drawing a straight boundary line through the scattered data points. The first split ("hours $\geq$ 4") is one such boundary, cutting the full rectangle of data points into two regions. Within the "hours $<$ 4" region, a second boundary ("attended recovery class") further divides that region. Each boundary corresponds to one additional level of the tree. Stopping before adding a third boundary — which would resolve the final 50/50 node — keeps the tree at depth 2, deliberately trading a small amount of training accuracy for a shallower, less complex tree.

Overfitting and underfitting

Overfitting: continuing to add branches/depth purely to correctly classify every remaining training instance, including single, ambiguous cases. This raises the complexity of the tree and the extra branches may not generalize to new data. Increasing `max_depth` on the study-hours tree can push training accuracy to 100%, but this is not desirable since it fits the training data overly precisely.

Underfitting: stopping the tree too early (too small a depth), leaving a large fraction of the training data incorrectly classified — for example, stopping at depth 1 where a further split would have resolved several more instances correctly.

Trade-off: the goal is an optimal depth between the two extremes. There is no formula for the "correct" depth — it's determined by hit-and-trial, guided by developer experience and data set size. For a data set of 10,000 instances, depth 2 would be recognized as far too shallow. A developer might start around depth 4 or 5 and increase it while performance keeps improving meaningfully — for example, from 60% training accuracy at depth 5, to 80% at depth 6, to 90% at depth 7 — stopping once further depth stops being worth the added complexity, deliberately not chasing 100% training accuracy.

Entropy vs. Gini

Gini impurity and entropy are both metrics used to decide tree splits; scikit-learn supports both and defaults to Gini (`criterion='gini'`). They move in opposite numeric directions:

Gini measures impurity and decreases toward 0 as the tree goes deeper and nodes become purer. Entropy measures information gained and increases toward a maximum of 1 as the tree goes deeper and more information is gained about the class.

Neither metric is inherently superior — both are valid measures of the same underlying idea (how mixed or resolved a node's classes are), expressed with different formulations and terminology.

Decision tree regression

By default, a decision tree is a classification algorithm — it assumes the output (y) variable is discrete/categorical. It can be adapted for regression problems (where y is continuous, such as predicting salary) by converting the continuous target into threshold-based categorical labels. For example, salaries above 50,000 might be labeled "high" and salaries below 50,000 labeled "low," with the tree then predicting which labeled range a new instance falls into rather than an exact numeric value. This adapted approach is called decision tree regression. It differs fundamentally from linear regression, which directly predicts an exact numeric value (such as an exact salary) rather than a labeled range.

Explainability

A key strength of decision trees is explainability. The entire decision path — which feature, which threshold, in what order — is visible, so it is possible to show anyone why a given instance was classified the way it was. This works even for someone without a mathematics or machine learning background, simply by walking them through the tree to the relevant leaf node. This is one reason decision trees are a preferable choice when the data set is small. By contrast, algorithms like logistic regression and especially neural networks are much harder to explain. Their learned weight parameters and the resulting fitted function are difficult to interpret intuitively, particularly for anyone who is not the model's own developer — often essentially impossible to explain to a non-technical stakeholder.

Building and training the tree in scikit-learn

The implementation uses a handful of familiar imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
```

`DecisionTreeClassifier` is the model class used to build the tree. `plot_tree` is a scikit-learn utility used purely to visualize the resulting tree structure — not required for training or prediction, but useful for inspecting how the algorithm split the data. `accuracy_score` computes the accuracy of the fitted model's predictions. The plot size is configured with `plt.rcParams['figure.figsize'] = (7, 4.5)`, and a random state of 42 is used consistently wherever randomness could otherwise vary results across runs — the same reproducibility approach used in the earlier logistic regression analysis.

A pandas DataFrame is constructed to match the study-hours example, with columns for hours of study, whether the recovery class was attended (0/1), and the pass/fail class label. The `hours` and `attended` columns become the input features (X); the pass/fail column is the target (y):

```
clf = DecisionTreeClassifier(criterion='gini', max_depth=2, random_state=42)
clf.fit(X, y)

predictions = clf.predict(X)
training_accuracy = accuracy_score(y, predictions)
print(f"Training Accuracy: {training_accuracy:.2f}")
```

`criterion='gini'` explicitly selects Gini impurity as the split metric (also scikit-learn's default, but set explicitly here for clarity; `criterion='entropy'` is the alternative).
`max_depth=2` limits the tree to two levels below the root, matching the by-hand example — chosen because the data set has only 8 instances.
`random_state=42` fixes any internal randomness for reproducibility.

This produces a training accuracy of 0.88 (88%) — one instance out of eight is misclassified, matching the by-hand result where the "attended = 1, hours < 4" leaf could not be perfectly resolved at depth 2.

Visualizing and reading the plotted tree

```
fig, ax = plt.subplots(figsize=(10, 6))
plot_tree(
    clf,
    feature_names=['hours', 'attended'],
    class_names=['fail', 'pass'],
    filled=True,
    rounded=True,
    fontsize=10,
    ax=ax
)
plt.title("Decision Tree")
plt.show()
```


The resulting tree matches the by-hand derivation, with one presentational difference: scikit-learn phrases the root condition as `hours <= 4.0` and routes instances satisfying this condition to the left, whereas the by-hand walkthrough used `hours >= 4` routing to the right. The underlying split and resulting classes are identical — only the true/false direction convention differs. Reading the plotted tree:

Root node: Gini = 0.469 (matching the by-hand root calculation), majority class = pass (5 pass vs. 3 fail out of 8, so when nothing else is known, the majority-class guess is "pass").

Right child (`hours > 4`): 4 instances, value = [0 fail, 4 pass], Gini = 0.0 — pure node, class = pass.

Left child (`hours ≤ 4`): 4 instances, 3 fail / 1 pass, Gini = 0.375. This branch splits further on `attended ≤ 0.5`, scikit-learn's way of expressing "attended = 0," since $0 \leq 0.5$ is true and $1 \leq 0.5$ is false — functionally identical to using `attended == 1` vs. `attended == 0`.

`attended = 0`: 2 instances, both fail, Gini = 0.0 — pure node, class = fail.

`attended = 1`: 1 pass / 1 fail, Gini = 0.5 — impure, tied node. scikit-learn still must assign some majority class here even though it's a genuine 50/50 tie. Whichever class is picked, exactly one instance at this leaf will be misclassified — effectively an arbitrary guess by the model at a tied node.

Manual (from-scratch) implementation of Gini impurity

To reinforce the underlying formula — purely for understanding, since scikit-learn's built-in implementation is what's used in practice — a Gini function and a weighted-Gini-after-split function can be written directly in Python:

```
def genie(labels):
    labels = np.array(labels)
    n = len(labels)
    if n == 0:
        return 0
    impurity = 1
    for cls in np.unique(labels):
        p = np.sum(labels == cls) / n
        impurity -= p ** 2
    return impurity
```

This takes the array of class labels reaching a branch, converts it to a NumPy array, and returns 0 immediately if no instances reached that branch (no data means no possible impurity). Otherwise it starts `impurity` at 1 (the theoretical maximum) and, for each unique class present, computes that class's proportion within the branch, squares it, and subtracts it — directly implementing $\text{Gini} = 1 - \sum p(\text{class})^2$. Calling this on the full label column (no split applied) reproduces the root-node Gini of 0.469, since 3 of 8 instances are "fail" and 5 of 8 are "pass."

A second function computes the weighted Gini for a candidate split condition (for example, `hours >= 4`) applied to the full DataFrame:

```
def weighted_genie_after_split(df, condition):
    left = df[~condition]
    right = df[condition]
    n = len(df)
    g_left = genie(left['pass'])
    g_right = genie(right['pass'])
```

```
weighted = (len(left) / n) * g_left + (len(right) / n) * g_right
return g_left, g_right, weighted
```

This splits the DataFrame into left and right subsets based on the condition, calls `genie` separately on each subset's labels, then combines the two resulting Gini values into one weighted score, using each branch's share of the total instance count as its weight. Applying this to `hours >= 4` reproduces `g_left = 0.375`, `g_right = 0.0`, and `weighted = 0.1875`, matching the by-hand calculation. Applying it to `attended == 1` as an alternative first split gives a weighted Gini of `0.438`, confirming that splitting on hours first is the better choice.

3. Key Takeaways

A decision tree is rule-based, not learning-based — it builds a nested if-else structure directly from the data instead of optimizing weight parameters, which makes it fast and highly explainable but potentially weaker than learning-based algorithms on huge data sets.

Gini impurity ($1 - \sum p(\text{class})^2$) measures how mixed a node's classes are, ranging from 0 (pure) to 0.5 (perfectly mixed, binary case). Every split is chosen to minimize the weighted Gini across its resulting branches.

Growing a tree deeper always reduces training error, but depth is deliberately capped — the trade-off between overfitting (too deep) and underfitting (too shallow) is resolved by hit-and-trial, not a fixed formula.

The scikit-learn implementation of `DecisionTreeClassifier` reproduces the by-hand Gini math exactly; the only difference is a flipped left/right convention (`hours <= 4.0` vs. the by-hand `hours >= 4`). The from-scratch Python implementation of Gini matches it too.

Decision trees are multi-class by nature and can be adapted to regression by converting a continuous target into threshold-based categorical labels.

Mental model: Think of a decision tree as a flowchart of yes/no questions built by picking, at each step, the question that splits the remaining data into the cleanest possible groups — stopping once asking one more question stops being worth the added complexity.